

PyTorch from Scratch

Concept

Tiny board

Computation

Interprete output

One-line story

Numbers become tensors. Tensors have shape, dtype and device. PyTorch tracks gradients and updates model weights with a training loop.

Learning route: tensors -> shapes -> gradients -> training loop

Why PyTorch after sklearn

Classical ML vs visible training machinery

Tensor ranks

scalar, vector, matrix, image batch

Shape tools

view, reshape, squeeze, unsqueeze, broadcasting

Training loop

forward -> loss -> zero_grad -> backward -> step

Why move from sklearn to PyTorch?

sklearn

- Ready-made models
- Quick baselines
- Clean `.fit()` / `.predict()` API
- Training details mostly hidden

PyTorch

- Custom neural networks
- GPU workflow
- Inspect weights and gradients
- Write the training loop yourself

Same linear idea, but PyTorch exposes model weights + gradients + optimiser.

Tensor mental model: shape, dtype, device

shape

How many axes?
How long each axis?

dtype

What kind of
number?

device

CPU or GPU
location

grad

Track
derivatives?

A tensor is not just values. It also carries information about layout and computation.

Baby version: a box of numbers with a label saying shape, type and place.

Scalar: a 0-dimensional tensor

scalar shape = `torch.Size([])`



one number
no row, column or axis

```
x = torch.tensor(42.0)
x.shape # torch.Size([])
x.ndim  # 0
x.item() # Python float
```

Where you see it

Loss values are usually scalars.

One number summarises many errors.

Vector: a 1-dimensional tensor

vector shape = [N]

[1.0, 2.0, 3.0, 4.0, 5.0]

Key idea

A vector is a flat list. It has length, but not “row” or “column”.

Operations: elementwise add/multiply, sum, mean, norm and dot product.

$$\text{dot}(a,b)=a_1b_1 + a_2b_2 + \dots + a_nb_n$$

Matrix: rows, columns and real datasets

matrix shape = [rows, cols]

Dataset view

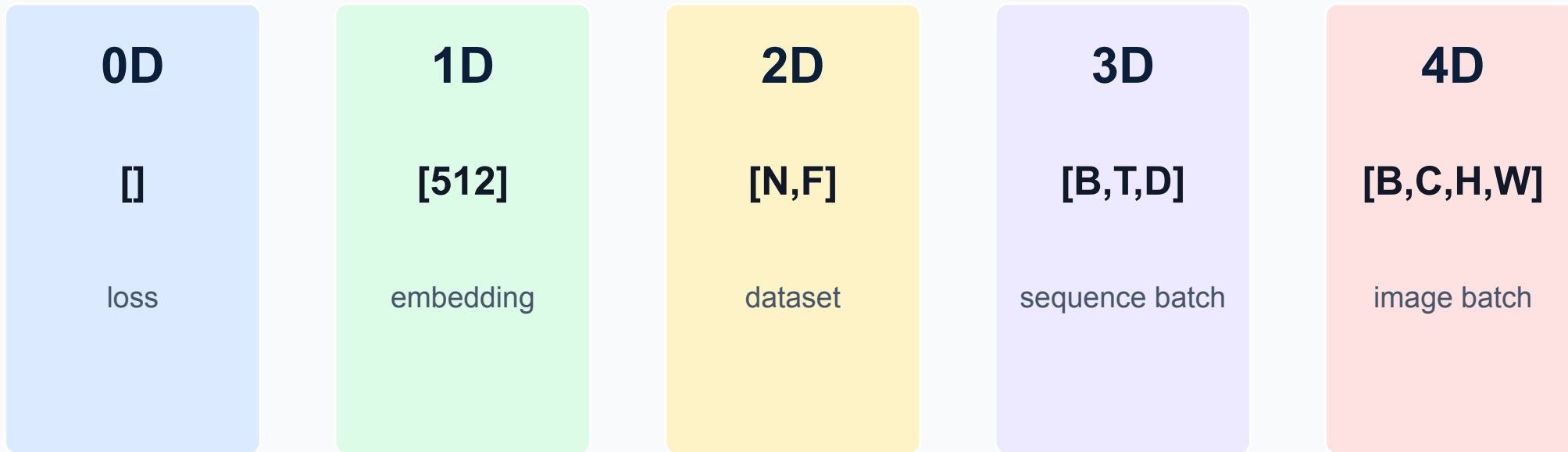
[100, 4] means
100 samples and 4 features.
This is ML dataset language.

```
M[1,2] # row 1, col 2
M[0,:] # row 0
M[:,1] # column 1

A * B  # elementwise
A @ B  # matrix multiplication
```

A @ B is the real linear-algebra operation.

Tensor rank ladder: scalar -> image batch



Teaching trick: tensor = N-dimensional array. It is not mysterious once you can read the shape.

Images in PyTorch: [batch, channels, height, width]

`images.shape = [32, 3, 64, 64]`

B

32 images
in one batch

C

3 colour
channels RGB

H

64 pixels
high

W

64 pixels
wide

```
images[0]           -> one image  
images[0,0]         -> one channel  
images[0,0,10,10]   -> one pixel
```

Shape confusion: (5,) vs (5,1) vs (1,5)

(5,)

No row/column concept
Shape has one number

(5,1)

Five rows
One column

(1,5)

One row
Five columns

Same 5 values, different behaviour in matrix multiplication and broadcasting.

Why the shape confusion matters

[5,1] @ [1,5] -> [5,5]

outer product

[1,5] @ [5,1] -> [1,1]

dot product stored as a matrix

torch.dot(a,a)

with [5] and [5] returns scalar []

```
sklearn target:  y.shape = [10]
PyTorch output: pred.shape = [10,1]
Fix: y.unsqueeze(1)  -> [10,1]
```

Common shape cheat sheet

[N]

labels, bias terms

[N,F]

feature matrix: N samples, F features

[N,1]

single regression output column

[N,C]

class logits for C classes

[B,T,D]

sequence batch

[B,C,H,W]

image batch, PyTorch NCHW

Shapes are the grammar of PyTorch. If grammar is wrong, the sentence will fail.

Reshaping: same values, new layout

24 numbers can be arranged as:

[24]

[6,4]

[2,3,4]

[8,3]

```
x = torch.arange(24)
x.view(6,4)
x.view(2,3,4)
x.view(-1,4)  # infer first dimension
```

Meaning of -1

“You calculate this dimension for me, as long as total elements stay the same.”

squeeze, unsqueeze and broadcasting

unsqueeze

Add a size-1 dimension
 $[3,5] \rightarrow [1,3,5]$

squeeze

Remove size-1 dimensions
 $[3,1,5,1] \rightarrow [3,5]$

broadcast

Expand compatible axes
without copying values

Broadcasting rule: align from the right; dimensions must match or one must be 1.

$[3,5]+[5] \rightarrow [3,5]$ $[3,5]+[3,1] \rightarrow [3,5]$ $[3,5]+[4] \rightarrow \text{error}$

dtype: what kind of numbers?

float32

Default for model inputs
and trainable weights.
Needed for gradients.

int64

Usually used for
labels, indices and
class IDs.

bool

Used for masks,
conditions and
True/False tensors.

```
x = torch.zeros(3, dtype=torch.float16)
z = a.to(torch.float64)
z2 = a.int()
```

**Wrong dtype can break a loss function even
when shape is correct.**

device: CPU or GPU location

CPU

Always available. Enough for this tutorial and many small demos.

CUDA / GPU

Useful for large tensors, neural networks and parallel computation.

```
device = "cuda" if torch.cuda.is_available() else "cpu"  
x = x.to(device)  
x.cpu().numpy()
```

Rule

All tensors in one operation must live on the same device.

Autograd: how PyTorch learns gradients

`requires_grad=True`

forward pass

scalar loss

`loss.backward()`

gradient = “which direction should each weight move to reduce loss?”

```
w = torch.tensor(3.0, requires_grad=True)
loss = (y - target)**2
loss.backward()
w.grad, b.grad
```

backward computes gradients. The optimiser step updates parameters.

Putting it together: sklearn -> PyTorch

sklearn baseline

LinearRegression hides the training algorithm.
You call fit and predict.
Fast and concise.

PyTorch equivalent

You build tensors, model, loss, optimiser and loop yourself.
More control.

data

baseline

tensors

model

PyTorch model pieces: data, model, loss, optimiser

Tensors

X_tr, y_tr
float32 tensors

Model

nn.Linear(8,1)
learns weight + bias

Loss

nn.MSELoss()
scalar error

Optimiser

SGD updates
parameters

```
model = nn.Linear(8, 1)
loss_fn = nn.MSELoss()
optimizer = torch.optim.SGD(model.parameters(), lr=0.01)
```

Parameters

model.weight shape [1,8]
model.bias shape [1]

The PyTorch training loop

forward
`y_pred=model(X)`

loss
compare truth

zero_grad
clear old grads

backward
compute grads

step
update weights

Why zero_grad?

Gradients accumulate by default. Clearing prevents accidental mixing of old and new signals.

Why step?

The optimiser uses gradients to change parameters, hoping the next loss is smaller.

forward -> loss -> zero_grad -> backward -> step

Evaluation: `model.eval()` and `torch.no_grad()`

`model.eval()`

Switch model into evaluation mode.
Important for dropout/batchnorm in larger models.

`torch.no_grad()`

Do not track gradients.
Saves memory and compute.

Plot 1

Training loss over epochs should generally go down.

Plot 2

Predicted vs actual: points near diagonal mean good predictions.

Bonus: why PyTorch scales beyond sklearn

```
deep_model = nn.Sequential(  
    nn.Linear(8,64), nn.ReLU(),  
    nn.Linear(64,32), nn.ReLU(),  
    nn.Linear(32,1)  
)
```

What ReLU does

Adds nonlinearity so stacked layers can model richer relationships.

Why Sequential?

Chains layers in order: input -> layer -> activation -> output.

More layers = more expressiveness, but also more responsibility.